Processador MIPS Monociclo com Instruções de Desvio

Segunda Avaliação: Implementação em Hardware e Síntese em FPGA

José Dercy Vieira da Silva Filho
Vinicius Menezes
Isabela Furtado

Faculdade de Engenharia Elétrica e da Computação – FEEC
Universidade Federal do Amazonas – UFAM

Arquitetura de Sistemas Digitais e Laboratório

Objetivos da Segunda Entrega

Escopo do Projeto

Implementação e validação completa do processador **MIPS Monociclo de 32 bits** em Verilog HDL, sintetizável para FPGA.

Principais Metas desta Entrega:

- **Caminho de Dados com Saltos:** Suporte completo a desvios condicionais (`beq`) e incondicionais (`j`, `jal`, `jr`).
- **Síntese em FPGA:** Prototipagem na placa **DE10-Lite** (Intel MAX 10 `10M50DAF484C7G`).
- **Simulação RTL:** Validação comportamental de todos os módulos via Icarus Verilog e GTKWave.
- **Código 100% Parametrizado:** Flexibilidade de barramentos de dados e endereços.

Visão Geral da Arquitetura

O processador é estruturado em **4 grandes subsistemas** interconectados:

1. Caminho de Dados (Datapath)

Executa as instruções: PC, somadores, multiplexadores, Banco de Registradores e ULA.

2. Unidade de Controle

Decodifica `opcode` e `funct`, orquestrando os sinais de controle do hardware.

3. Memória de Instruções (ROM)

Armazena o código de máquina do programa a ser executado; leitura combinacional.

4. Memória de Dados (RAM)

Armazena variáveis em tempo de execução; leitura assíncrona (`lw`) e escrita síncrona (`sw`).

Modelo Harvard: Barramentos separados para código e dados evitam conflitos de acesso no monociclo.

O Caminho de Dados (Datapath)

Função Principal

Roteamento de dados entre registradores, operadores aritméticos e memórias em um único ciclo de clock.

Componentes Fundamentais:

- **PC (Program Counter):** Registrador de 32 bits atualizado a cada borda de subida do clock.
- **Somador do PC:** Calcula sequencialmente $\text{PC} + 4$ para a próxima instrução.
- **Extensor de Sinal:** Converte imediatos de 16 bits para 32 bits com preservação de sinal.
- **Somador de Branch:** Calcula o endereço alvo: $(\text{PC} + 4) + (\text{offset} \ll 2)$.
- **Multiplexadores em Cascata:** Decidem o endereço do próximo PC e as fontes de dados da ULA e registradores.

Suporte aos Saltos Incondicionais

O processador implementa os 3 modos de salto da arquitetura MIPS:

- **`j target` (Jump Incondicional):**
  - Salto direto para endereço pseudo-absoluto.
  - Equação de endereço: PC $\leftarrow \{\text{PC} + 4[31:28],\ \text{target}[25:0],\ 2'\text{b00}\}$.
- **`jal target` (Jump and Link):**
  - Chamada de função: executa o salto e salva o endereço de retorno.
  - Salva PC $+ 4$ no registrador especial **$ra ($31)**.
- **`jr $rs` (Jump Register):**
  - Retorno de procedimento carregando o endereço contido em registrador.
  - Equação de endereço: PC $\leftarrow$ Reg$[rs]$ (normalmente `jr $ra`).

Lógica de Seleção do Próximo PC

Para suportar sequenciamento normal, branch e jumps no mesmo ciclo, foi utilizada uma **cascata de multiplexadores 2:1**:

1. **1º Estágio (Branch):**
   Seleciona entre $\mathrm{PC} + 4$ e $\mathrm{PC}_{\mathrm{branch}}$ via controle:

   $$\mathrm{pc_src} = \mathrm{branch} \;\&\; \mathrm{zero}$$

2. **2º Estágio (Jump Direto e JAL):**
   Seleciona entre o resultado pós-branch e $\mathrm{PC}_{\mathrm{jump}}$ via sinal `jump`.

3. **3º Estágio (Jump Register - JR):**
   Seleciona entre o resultado anterior e o dado do registrador `read_data1` via sinal `jr`.

Hierarquia: O salto indireto (`jr`) possui a prioridade final no fechamento do ciclo.

Unidade Lógica e Aritmética (ULA)

Características da ULA de 32 bits

Módulo combinacional parametrizado responsável por todas as operações de dados.

Operações Implementadas:

- Lógicas: `AND`, `OR`, `NOR`.
- Aritméticas: Adição (`add`) e Subtração (`sub`).
- Comparação: `SLT` (*Set on Less Than*) com tratamento de sinal via `$signed`.

Destaques de Projeto

- **Somador Compartilhado:**
  Subtração calculada como $A + \bar{B} + 1$, poupando área física na FPGA.
- **Flag Zero Rápida:**
  Detecção por redução unária:

$$\text{zero} = \sim | \,\text{alu_result}$$

Unidade de Controle Integrada

Orquestra o caminho de dados dividindo a tarefa em dois blocos combinacionais:

`main_control`

Decodifica o opcode de 6 bits:

- Define sinais para Tipo-R, `lw`, `sw`, `beq`, `j` e `jal`.
- Controla `reg_dst`, `alu_src`, `mem_to_reg`, etc.

`alu_control`

Decodifica o campo `funct`:

- Gera o controle de 4 bits `alu_ctrl` para a ULA.
- Detecta a instrução de retorno `jr` (`funct = 6'b001000`).

Trava de Segurança Crítica do JR

```
assign reg_write = reg_write_wire & ~jr;
```

Impede que a instrução `jr` sobrescreva o registrador **$ra** com lixo da ULA.

Banco de Registradores e Memórias

- **Banco de Registradores (`reg_file`):**
  - 32 registradores de 32 bits com 2 portas de leitura combinacionais e 1 de escrita síncrona.
  - **Regra MIPS:** Registrador **$0 ($zero)** é permanentemente travado em zero.
- **Memória de Instruções (`instr_mem` – ROM):**
  - Leitura por palavra indexada por `PC[31:2]` (divisão por 4 sem custo em hardware).
  - Carga do código de máquina via diretiva `$readmemh`.
- **Memória de Dados (`data_mem` – RAM):**
  - Leitura combinacional para `lw` e escrita síncrona habilitada por `mem_write` para `sw`.

Síntese e Prototipagem em FPGA (Rubrica 1)

Configuração do Projeto Quartus Prime

- **Placa Alvo:** Terasic DE10-Lite com FPGA **Intel MAX 10 (10M50DAF484C7G)**.
- **Arquivos de Projeto:** `mips_monociclo.qpf` e `mips_monociclo.qsf`.

Resultados da Compilação no Quartus:

- **Análise e Elaboração:** Concluída com **0 erros**.
- **Hierarquia Completa:** Todos os 15 módulos HDL reconhecidos e mapeados.
- **Inspeção RTL:** Diagrama esquemático verificado via *RTL Viewer*, comprovando a ausência de latches espúrios e barramentos desconectados.
- **Automação:** Alvo `make quartus` integrado ao Makefile do projeto.

Simulação RTL e Validação Funcional (Rubrica 3)

Fluxo de Testes Automatizado

Compilação e execução via **Icarus Verilog** (`iverilog/vvp`) com inspeção no **GTKWave**.

Testbenches Unitários

- **ULA (`tb_alu.v`):** 11 testes aprovados (100%) cobrindo aritmética, lógica e `slt`.
- **Controle (`tb_control_unit.v`):** 13 testes aprovados (100%) cobrindo Tipo-R, saltos e desvios.

Testbench Integrado (Top)

- Execução de programa com saltos `j`, `jal` e `jr`.
- Verificação do salvamento correto de PC+4 em **$ra**.
- Formas de onda salvas em arquivo `.vcd` para demonstração.

Conformidade com as Rubricas de Avaliação

Rubrica	Requisito Oficial	Status
Rubrica 1	Configuração do ambiente de desenvolvimento e síntese no Quartus Prime	**100% Concluído**
Rubrica 2	Caminho de dados completo com suporte a Jumps (`j`, `jal`, `jr`)	**100% Concluído**
Rubrica 3	Validação de todos os componentes individuais via testbenches RTL	**100% Concluído**
Rubrica 4	Código 100% parametrizado (`parameter DATA_WIDTH = 32`)	**100% Concluído**
Rubrica 5	Apresentação em vídeo (YouTube) e relatório técnico padrão IEEE	**100% Concluído**

Referências Bibliográficas

- **HARRIS, D. M.; HARRIS, S. L.** *Digital Design and Computer Architecture*. 2nd ed. Morgan Kaufmann, 2013.
  - Arquitetura MIPS monociclo, somadores dedicados e ULA compartilhada.
- **PATTERSON, D. A.; HENNESSY, J. L.** *Computer Organization and Design: The Hardware/Software Interface*. 5th ed. Morgan Kaufmann, 2014.
  - Conjunto de instruções MIPS32 e decodificação da Unidade de Controle.
- **TERASIC TECHNOLOGIES.** *DE10-Lite User Manual: Intel MAX 10 FPGA*. Terasic Inc., 2016.
  - Configuração de pinagem física e síntese no Intel Quartus Prime.
- **IEEE Std 1364-2001.** *IEEE Standard for Verilog Hardware Description Language*. IEEE, 2001.
  - Padrão Verilog-2001 e diretrizes de síntese de hardware.

Conclusão e Encerramento

Síntese dos Resultados

- Processador MIPS Monociclo totalmente funcional e integrado em Verilog HDL.
- Suporte bem-sucedido a chamadas e retornos de subrotinas através de `jal` e `jr`.
- Projeto preparado e testado para gravação do bitstream na FPGA Intel MAX 10.

Obrigado pela atenção!

Equipe: José Dercy Vieira da Silva Filho · Vinicius Menezes · Isabela Furtado
Universidade Federal do Amazonas – UFAM / FEEC